



Marketplace Content Pack — Troubleshooting & Data Provenance

The official KOI pack from demisto/content — Packs/Koi, version 1.2.3, 13 commands, integration only.

READ THIS FIRST Two different content packs are called KOI. Both use integration id "KOI", both are category Endpoint, both are authored by "Cortex XSOAR", both use koi-* commands. This document describes ONLY the Marketplace pack, version 1.2.3. It does not apply to the custom in-house pack (v1.3.0, 26 commands), which is a different implementation.

Document	KOI Marketplace Pack — Troubleshooting & Data Provenance Guide v1.0
Pack	KOI — demisto/content / Packs/Koi/Integrations/Koi/Koi.yml
Pack version	1.2.3
Integration	KOI (id KOI), fromversion 6.10.0
Docker image	demisto/fastapi:0.125.0.10158186
Marketplaces	xsoar, marketplacev2, platform
Command surface	13 commands · 67 arguments · 131 output declarations
Source pinned at	md5 5497cdddedeb0c0d7d0b371aa075a64c, byte-identical to master on 2026-07-20
Live verification	Tenant api-ayman.xdr.eu.paloaltonetworks.com, instances KOI_PAET and KOI_PLTS, 20–21 July 2026. API-side only: the API behind 8 of the 13 commands was exercised read-only; the endpoints behind the 5 state-changing commands were not called, and no koi-* command was executed through XSIAM (section 3)
Companion pack	KOI Content Extension (KoiContentExtension) v1.0.0 — SEPARATE, additional content. Not part of this pack. See the note below.

OPTIONAL COMPANION PACK — SEPARATE, ADDITIONAL CONTENT A second, separate content pack, KoiContentExtension ("KOI Content Extension", v1.0.0, community-supported), is published alongside this one. It ships NO integration and NO commands of its own; it adds parsing rules, modeling rules, 10 playbooks and a dashboard on top of the KOI integration, normalising and modelling the koi_koi_raw dataset this pack produces. It is NOT part of the Marketplace KOI pack, and nothing in this troubleshooting guide depends on it. Where a fix below names a "KOI Ext" playbook, that is this companion pack, flagged as such.

Contents

How to Read This Guide

1. Identifying the Pack Before You Troubleshoot Anything

- 1.1 The command surface is the fingerprint
- 1.2 Commands that do not exist here
- 1.3 Context prefixes, and where they collide
- 1.4 Commands you cannot branch on
- 1.5 Five commands change state; the pack flags only two of them

2. The Data Path on This Tenant

- 2.1 Instance configuration as verified
- 2.2 What is in the dataset

Part A — Marketplace Pack Troubleshooting

- A1. Commands work but the dataset is empty
- A2. Events arrive but every XDM field is empty
- A3. XQL against resources or observables returns nothing
- A4. alert_type is always null
- A5. view=browser_extensions returns HTTP 400
- A6. koi-inventory-search fails on its filter
- A7. Two instances, one dataset, no way to tell them apart
- A8. The instance runs through an engine — your reachability test is testing the wrong path
- A9. Allowlist and blocklist "return nothing" — checking total_count
- A10. Context from one inventory command disappears after the next one runs
- A11. Alert counts look inflated — the same alert appears hundreds of times
- A12. A marketplace value from an event is rejected with HTTP 400
- A13. A host stops producing events — KOI is run-on-demand, and nothing scanned it
- A14. You installed something but KOI doesn't see it

3. Environment Finding — Automating Command Tests on This Tenant

Part B — Endpoint Findings Inherited from the Custom-Pack Investigation

- B1. There is no resident KOI agent on Windows
- B2. The file map — the four files recorded under C:\ProgramData\Koi
- B3. Only a fresh mtime proves an authenticated round trip
- B4. Extension display names come from _locales, not the manifest
- B5. Installed software needs both registry hives
- B6. KOI bundles its own Python
- B7. The powershell -Command - trap
- B8. Install path decides event collection

4. Fast Triage Order

5. Command Reference — Generated From the Pack

Appendix — Provenance of This Document

Section 5 lists all 13 commands individually; those headings are omitted here.

How to Read This Guide

Every load-bearing claim in this guide is traceable to one of the four inputs listed in the appendix, and where the source matters it is marked with an inline evidence tag. Tags are applied by hand and are not applied to every sentence — the Symptom / Cause / Do this rows of Part A are advice composed from the tagged Evidence row directly beneath them, not independent claims. The tags mean:

Tag	Meaning
[YAML]	Extracted mechanically from the pinned pack YAML (md5 5497cdddedeb0c0d7d0b371aa075a64c) and rebuilt into this document at generation time. Not transcribed by hand.
[LIVE]	Observed on the tenant or against the KOI API on 20 July 2026. The observation is recorded in VERIFIED_FACTS.md, and the per-instance probe results in evidence/followup-probes.json.
[CODE]	Read from a pack source file that is NOT the integration YAML — Koi.py, or command_examples.txt. The YAML and the Python are different artefacts, so these are not tagged [YAML]. Read from source; not executed.
[PACK]	Established by inspecting the pack directory itself — what the pack does and does not contain (for example: it ships an integration only, no parsing rules and no modeling rules). That is a filesystem fact about the pack, not an observation of the tenant or the API, so it is not tagged [LIVE]; and the integration YAML carries no inventory of the pack's other contents, so it is not tagged [YAML] either.
[INHERITED]	Carried forward from the earlier custom-pack investigation. Pack-independent, but NOT re-verified in this session. Part B is entirely of this kind.
[DERIVED]	Composed here from tagged facts — typically a query string assembled from verified dataset and field names. The facts are verified; this exact string was not itself executed.
[UNVERIFIED]	Stated as unknown. Never asserted as fact.

The document has two parts, and they are kept visually distinct on purpose:

- **Part A** (orange headings) — troubleshooting specific to this Marketplace pack, on this tenant, grounded in this session's verification. Each block's Evidence row says whether the symptom was observed here or reasoned from evidence gathered here.
- **Part B** (blue headings, tinted background) — endpoint-side findings inherited from the custom-pack investigation. Pack-independent, not re-verified here.

CASING Context paths in this pack are inconsistent and must stay that way: events are written to KOI.Event.* (upper), everything else to Koi.* (mixed). "Correcting" either one breaks the DT paths in any playbook or layout that reads them.

1. Identifying the Pack Before You Troubleshoot Anything

Half of all KOI troubleshooting time is spent debugging the wrong pack. Establish which one is installed before anything else.

1.1 The command surface is the fingerprint

[YAML] This pack ships exactly 13 commands. If the instance you are looking at offers more than that — in particular anything device-centric — you are not on this pack.

Command	Args	Outputs	API call
koi-allowlist-get	0	9	GET /policies/allowlist
koi-allowlist-items-add	5	0	POST /policies/allowlist
koi-allowlist-items-remove	5	0	DELETE /policies/allowlist
koi-blocklist-get	0	9	GET /policies/blocklist
koi-blocklist-items-add	5	0	POST /policies/blocklist
koi-blocklist-items-remove	5	0	DELETE /policies/blocklist
koi-get-events	5	4	GET /alerts and /audit-logs
koi-inventory-item-endpoints-list	6	10	GET /inventory/{item_id}/endpoints
koi-inventory-item-get	3	27	GET /inventory/{item_id}
koi-inventory-list	21	27	GET /inventory
koi-inventory-search	7	27	POST /inventory/search
koi-policy-list	3	9	GET /policies
koi-policy-status-update	2	9	PUT /policies/{id}

[YAML] Base URL <https://api.prod.koi.security/> — that is the default of the pack's url parameter, and it is the only part of the API contract the YAML states. [CODE] The /api/external/v2 path suffix, the Authorization: Bearer <api_key> header and every endpoint path in the table above were read from Koi.py, not from the YAML (VERIFIED_FACTS \$4.1). [LIVE] Eight of those endpoints were then called directly and answered as described; the other five were not called at all — see the note below.

EIGHT OF THE THIRTEEN WERE EXERCISED — NOT ALL OF THEM The API behind 8 commands was called read-only on 20 July 2026: koi-policy-list, koi-allowlist-get, koi-blocklist-get, koi-inventory-list, koi-inventory-item-get, koi-inventory-item-endpoints-list, koi-inventory-search and koi-get-events. The 5 state-changing commands (koi-policy-status-update, koi-allowlist-items-add / -remove, koi-blocklist-items-add / -remove) were deliberately NOT run, because each one mutates tenant state. Their rows in the table above are the endpoint Koi.py would call, not an observation. Nothing in this guide asserts that all 13 commands were exercised.

"READ-ONLY" DESCRIBES HOW THEY WERE CALLED — SEVEN OF THE EIGHT ARE REPEATABLE, ONE IS NOT koi-get-events is a GET, but it is not safe to re-run casually. The pack's own description of it says to "use this command for development and debugging only, as it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism", and its should_push_events argument (default false) pushes the events it retrieves into XSIAM — into koi_koi_raw — when set to true. That argument is handled by the integration, not by the API: it is not a parameter of GET /alerts or /audit-logs, and no such push could have happened here, because what was exercised on 20 July 2026 was an HTTP request to those two endpoints, made directly and read-only. The command itself was never executed. Of the 8 non-mutating commands, 7 are freely repeatable and this one is not — do not put it in a "safe to repeat" list without that caveat.

1.2 Commands that do not exist here

These belong to the custom in-house pack only. On this pack they fail as unknown commands, and any playbook, layout or dashboard built on them is inert:

Not in this pack	Not in this pack
koi-devices-list	koi-device-inventory-get
koi-koidex-risk-report	koi-koidex-search
koi-remediations-list	koi-approval-requests-list
koi-findings-list	koi-users-list
koi-groups-list	koi-runtime-policies-list
koi-runtime-policy-get	koi-fetch-context-get
koi-fetch-context-set	

NO DEVICE PREFIX There is no Koi.Device.* context anywhere in this pack — not in the YAML, not in Koi.py, not in the README. The model is item-centric: endpoints are reached only from an item, through Koi.Inventory.Endpoint.*, which only koi-inventory-item-endpoints-list writes. If a query, layout or playbook reads Koi.Device.*, it will never resolve.

1.3 Context prefixes, and where they collide

[YAML] 131 output declarations span only 68 distinct context paths. The 63 of difference is a count of redundant declarations, not of paths: 36 distinct paths are declared by more than one command, and those commands overwrite each other's context.

Prefix	Paths	Declared by	Commands
KOI.Event	4	1	koi-get-events
Koi.Allowlist	9	1	koi-allowlist-get
Koi.Blocklist	9	1	koi-blocklist-get
Koi.Inventory	37	4	koi-inventory-item-endpoints-list koi-inventory-item-get koi-inventory-list koi-inventory-search
Koi.Policy	9	2	koi-policy-list koi-policy-status-update

Consequence: the last command to run wins — but only among commands that share a path. koi-inventory-item-get, koi-inventory-list, koi-inventory-search declare the same 27 Koi.Inventory item paths, and both policy commands declare the same Koi.Policy paths. In a playbook that calls two of them, read the context immediately after each call or copy it to a distinct key.

Not part of that: koi-inventory-item-endpoints-list appears in the table above under Koi.Inventory, but it writes only the nested Koi.Inventory.Endpoint.* paths, which no other command declares. It cannot overwrite the three item-level commands and they cannot overwrite it.

1.4 Commands you cannot branch on

[YAML] 4 commands declare no outputs at all — they return a war-room message only. A playbook cannot test their result:

- koi-allowlist-items-add
- koi-allowlist-items-remove
- koi-blocklist-items-add
- koi-blocklist-items-remove

[YAML] 2 commands are marked execution: true (potentially harmful) and will prompt or be blocked by policy accordingly: koi-allowlist-items-remove, koi-blocklist-items-remove

1.5 Five commands change state; the pack flags only two of them

[YAML] execution: true is set on koi-allowlist-items-remove and koi-blocklist-items-remove only. [CODE] But 5 of the 13 commands mutate tenant state — that is read from the HTTP method each one issues in Koi.py (PUT, POST or DELETE), not from having watched any of them run: none of the five was executed here. The other 3 carry no flag at all, so they do not get the confirmation treatment execution: true triggers.

Command	execution: true	What it does
koi-policy-status-update	no — unflagged	PUT /policies/{id} — changes governance state
koi-allowlist-items-add	no — unflagged	POST /policies/allowlist — changes governance state
koi-allowlist-items-remove	yes	DELETE /policies/allowlist — changes governance state
koi-blocklist-items-add	no — unflagged	POST /policies/blocklist — changes governance state
koi-blocklist-items-remove	yes	DELETE /policies/blocklist — changes governance state

DO NOT READ "NOT FLAGGED" AS "SAFE" The two add commands and koi-policy-status-update change the tenant's allowlist, blocklist and policy state with no harmful-command flag. None of these five was run during verification, precisely because they mutate state — so this guide describes no observed behaviour for any of them. Treat "not flagged" as "not flagged".

2. The Data Path on This Tenant

Two things flow out of this integration and they travel by different routes. Knowing which one you are debugging saves the whole investigation.

```
commands: war room -> instance -> engine -> https://api.prod.koi.security/api/external/v2
events:   fetch loop -> instance -> engine -> /alerts + /audit-logs -> dataset koi_koi_raw
```

[YAML] The second path exists on XSIAM / platform only. The pack sets `isfetchevents: true` but overrides it with `isfetchevents:xsoar: false`, so on an XSOAR tenant there is no fetch loop and no dataset — see 2.1.

[LIVE] Both instances (KOI_PAET, KOI_PLTS) run through engine `c3664d21-63fb-4b2c-b16c-56cd547a3d79` with `propagationLabels ["all"]` — not direct tenant egress.

2.1 Instance configuration as verified

[YAML] Parameter names, types and defaults come from the pack. [LIVE] The right-hand column is what both instances actually carry.

Parameter	Required	Pack default	Live value on both instances
<code>url</code>	yes	<code>https://api.prod.koi.security/</code>	<code>https://api.prod.koi.security/</code>
<code>api_key</code>	yes	—	set (per instance, different keys)
<code>insecure</code>	no	—	<code>false</code>
<code>proxy</code>	no	—	<code>false</code>
<code>isFetchEvents</code>	no	—	<code>true</code>
<code>event_types_to_fetch</code>	yes	<code>Alerts,Audit</code>	<code>["Alerts", "Audit"]</code>
<code>audit_types_filter</code>	no	—	<code>[]</code> — no filter, all audit types
<code>max_fetch</code>	no	<code>5000</code>	<code>5000</code>
<code>eventFetchInterval</code>	no	<code>1</code>	<code>1</code> (minute)

[YAML] The 5 Collect-section parameters are hidden on the xsoar marketplace (hidden: `["xsoar"]` in the pack). They only appear when the pack is installed from a marketplace target that supports event collection — see A1.

EVENT COLLECTION IS XSIAM / PLATFORM ONLY The YAML sets `isfetchevents: true` and then overrides it on the very next line with `isfetchevents:xsoar: false`. Every Collect parameter is likewise hidden: `["xsoar"]`. On an XSOAR tenant this pack is a command integration and nothing else — no fetch loop, no dataset. `pack_metadata.json` still lists xsoar among its marketplaces, so the pack installs there and simply does not collect. Never say this pack "ships an event collector" without that qualifier.

2.2 What is in the dataset

[LIVE] Over 20 June – 20 July 2026:

Measure	Value	Note
Dataset	<code>koi_koi_raw</code>	Confirmed to exist and be populated — no longer an inference
Events	20,156	30-day window, this tenant
Distinct hostnames	80	
<code>_vendor / _product</code>	<code>koi / koi</code>	Consistent with <code>send_events_to_xsiam(vendor="koi", product="koi")</code> in <code>Koi.py</code>
<code>source_log_type = Audit</code>	19,842	Flat, KOI-native schema

Measure	Value	Note
source_log_type = Alerts	314	OCSF schema — class_uid 2007, type_uid 200701, is_alert true

ONE DATASET, TWO SCHEMAS Audit rows and Alert rows share koi_koi_raw but have nothing in common. All OCSF fields are null on Audit rows and all Audit fields are null on Alert rows. Always filter on source_log_type first — a query that does not is averaging two unrelated tables.

Part A — Marketplace Pack Troubleshooting

Fourteen failure modes. Each block gives the symptom as it presents, the cause, the evidence behind the diagnosis, and what to do. The Evidence row is the tagged part; the other three rows are advice composed from it.

They are not all of one kind, and the difference matters:

- **Observed on this tenant on 20 July 2026** — A2, A3, A4, A5, A8, A9. The condition described was seen, in the dataset or against the live API.
- **Observed on this tenant on 21 July 2026** — A11, A12, A13 and A14. The Alerts-stream duplication, the event-vs-API marketplace vocabulary mismatch, the run-on-demand scan behaviour and the user-profile scan scope were each measured live on 21 July 2026 (VERIFIED_FACTS §7b–§7e). A13 also re-verifies first-hand a finding first established in the earlier custom-pack investigation (Part B, B1).
- **Part observed, part read from the integration source** — A6 and A7. Of A6's three failure modes, two were reproduced as HTTP responses on both instances; the third is a refusal inside Koi.py, read from the source and never executed, because the integration raises before any API call is made. In A7, both instances' configuration and both KOI tenants' volumes were read live, but that the two instances land in the same koi_koi_raw dataset could not be observed: the pack ships no field identifying the producing instance, so no row can be attributed to one. That half follows from the configuration and from `send_events_to_xsiam(vendor="koi", product="koi")` in Koi.py. In both blocks the Evidence row says which half is which.
- **Reasoned from verified evidence, not observed here** — A1 and A10. A1 describes an empty dataset, which is NOT the state of this tenant: koi_koi_raw exists and holds 20,156 events. A10 follows from the pack's own duplicate output declarations; the commands could not be run through XSIAM here (section 3), so the overwrite was never watched happening.

A1. Commands work but the dataset is empty

Symptom	koi-* commands return data in the war room. dataset = koi_koi_raw returns nothing, or the dataset does not exist.
Cause	Commands and event collection are independent. Commands go out on demand; events only flow if the instance is actually a collector. An instance installed from a marketplace target without event support has the Collect section hidden and isfetchevents effectively off — on an XSOAR tenant that is always the case, because the pack overrides isfetchevents to false there.
Evidence	NOT OBSERVED ON THIS TENANT — reasoned from the evidence below. [LIVE] Here both instances have Fetch events enabled and koi_koi_raw is populated with 20,156 events, so this symptom is a failure mode to recognise, not something seen here. [YAML] The pack declares isfetchevents: true but overrides it with isfetchevents:xsoar: false; marketplaces are xsoar, marketplacev2, platform and the five Collect parameters carry hidden: ["xsoar"], so on XSOAR there is no collector at all. [INHERITED] A pack zip built for the wrong marketplace target carries isfetchevents: false and ships no rules.
Do this	Open the instance. If there is no "Fetch events" toggle at all, the installed pack build is not a collector — reinstall from the correct marketplace target rather than editing the instance. If the toggle is present but off, enable it and wait one fetch interval.

Confirm collection from the query side before touching the instance again:

```
dataset = koi_koi_raw
| comp count() as events by source_log_type
```

[DERIVED] Query text composed from the verified dataset name and the verified discriminator field, and not executed in this form. [LIVE] On this tenant that breakdown was two rows, Audit far larger than Alerts. Judge your own result by that shape — two rows, one much larger — not by matching a count.

If the dataset does not resolve at all, nothing has ever been written — this is an instance problem, not a query problem. If it resolves but is empty for your time window, check the fetch interval (eventFetchInterval is 1 minute on this tenant) and widen the window.

A2. Events arrive but every XDM field is empty

Symptom	koi_koi_raw has rows, but xdm.* is null on all of them. Correlation rules, out-of-the-box XSIAM widgets and anything keyed on the Cortex Data Model match nothing.
Cause	Expected behaviour, not a fault. This pack ships an integration and nothing else — no parsing rules, no modeling rules. Nothing maps KOI events into XDM.
Evidence	[LIVE] No XDM fields are populated at all across the 30-day window on this tenant. [PACK] The pack directory ships an integration only — no parsing rules, no modeling rules (VERIFIED_FACTS §3.2). That second half is a filesystem fact about the pack, established by inspecting its contents: it is neither an observation of the tenant nor a line of the YAML, which is why it is tagged [PACK] and not [LIVE] or [YAML]. The custom pack's modeling rules are a different pack and do not apply here.
Do this	Query the raw fields. Do not wait for XDM to fill in — it never will unless you write and ship modeling rules yourself, which is additional content and outside this pack.

The raw fields that are actually populated, by schema:

source_log_type	Fields to query
Audit	type, action, category, object_name, object_type, hostname, item_version
Alerts	class_uid, type_uid, is_alert, severity_id, confidence_id, risk_level_id, status_id, resources, observables, metadata

[LIVE] Audit type values and counts observed over the 30-day window: extensions 16,579 · devices 2,971 · remediation 244 · policies 32 · approval_requests 8 · guardrails 6 · notifications 2. Observed category: system 16,825 · user 3,017. These are a snapshot of a dataset that is still being written to — read them as the shape of the distribution, not as figures your own query has to match.

```
dataset = koi_koi_raw
| filter source_log_type = "Audit"
| comp count() as events by type, category
| sort desc events
```

[DERIVED] Query composed from verified field names and not executed in this form. It is written to reproduce, on your own window, the breakdown recorded above — nothing is claimed here about what it returns. The figures above are the observation, taken from a dataset that is still being written to.

A3. XQL against resources or observables returns nothing

Symptom	A filter or comp on resources, observables or metadata silently returns zero rows — no error, no warning.
Cause	On alert rows these three are JSON strings, not objects. XQL treats them as opaque text, so any field-style access misses.

Evidence	[LIVE] Confirmed on alert rows in koi_koi_raw. The failure mode is silence, which is why it costs time.
Do this	Extract before you filter, with json_extract / json_extract_scalar.

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts"
| alter first_resource = json_extract(resources, "$[0]")
| alter first_observable = json_extract(observables, "$[0]")
| fields _time, severity_id, risk_level_id, status_id, first_resource, first_observable
```

[LIVE] That resources/observables/metadata are JSON strings is verified. [DERIVED] The query text above is composed from those verified field names and was not itself executed; adjust the JSON path to the element you need.

WHY THIS MATTERS A query that treats these as structured fields returns an empty result rather than an error, so it reads as "there are no matching alerts" instead of "this query cannot work". Never conclude absence from an empty result on these three fields until you have re-run it with json_extract.

A4. alert_type is always null

Symptom	Any rule, query, playbook condition or dashboard keyed on alert_type matches nothing, on any alert row.
Cause	The field is never populated by this pack's collection path.
Evidence	[LIVE] filter alert_type != null over the full 30-day window returns zero rows.
Do this	Key on what is populated instead: source_log_type to separate the schemas, then class_uid / type_uid / severity_id / risk_level_id / status_id on alert rows.

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts" and alert_type != null
| comp count() as n
```

[LIVE] The underlying check was made: filter alert_type != null over the full 30-day window returned zero rows. [DERIVED] The query text above is that check composed with source_log_type and a count, and was not itself executed in this form. It is written to be run as a check on your own window — nothing is claimed here about what it returns — and not to be built on as a working filter.

PORTING FROM THE CUSTOM PACK The custom pack's triage content branches on alert_type. Ported to this pack unchanged, every branch is dead and the failure is silent. Rewrite those conditions before porting anything.

A5. view=browser_extensions returns HTTP 400

Symptom	The value copied from the pack's own command_examples.txt is rejected by the API with HTTP 400.
Cause	The shipped example is wrong. browser_extensions is in neither the YAML dropdown nor the API's accepted set. ide_extensions and packages are rejected the same way — but, unlike browser_extensions, neither of those appears in the shipped example; they are simply values a reader might guess.
Evidence	[LIVE] All three return HTTP 400 against the live API, on both instances, and the 400 body states the accepted set, so the error is not opaque. [CODE] Only browser_extensions occurs in Packs/Koi/Integrations/Koi/command_examples.txt — that is a pack source file, not the integration YAML. [YAML] The dropdown offers 6 values. [LIVE] The API accepts 9, and every one of the 12 values in the table below was called individually on both instances (evidence/followup-probes.json).

Do this	Use one of the values the API actually accepts. Three of them are missing from the dropdown and must be typed by hand.
----------------	--

Every value, on both instances. The two count columns are `total_count` from the response body — a snapshot of a live inventory that grows, not a threshold to test against. What is stable is the HTTP status column:

view value	In dropdown	In API set	HTTP	KOI_PAET items	KOI_PLTS items	Note
agentic_ai	yes	yes	200	226	1,342	
ai_models	yes	yes	200	3	28	
all_items	no	yes	200	0	0	missing from the dropdown, must be typed by hand; accepted, but 0 rows on both tenants
browser_extensions	no	no	400	—	—	the one bad value in the shipped example
code_packages	yes	yes	200	2,315	2,572	
extensions	yes	yes	200	180	417	
ide_extensions	no	no	400	—	—	not in the shipped example
mcp_servers	no	yes	200	21	42	missing from the dropdown, must be typed by hand
os_packages	yes	yes	200	350	393	
packages	no	no	400	—	—	not in the shipped example
repositories	no	yes	200	15	77	missing from the dropdown, must be typed by hand
software	yes	yes	200	406	1,046	

[LIVE] All 12 values were probed one at a time on both instances — the 9 the API names in its own 400 body, plus the 3 a reader might plausibly guess. Every cell above is an observed HTTP response, taken from `evidence/followup-probes.json`; none of it is inferred from the 400 body, and no view value is left unprobed. Both instances returned the same status for every value. *all_items* is the one to watch: it is accepted and still returns nothing — HTTP 200 with `total_count` 0 on both tenants, which is a result, not an error. Omitting view entirely returned 3,447 items on KOI_PAET and 5,644 → 5,646 on KOI_PLTS, so to ask for everything, omit the argument rather than passing *all_items*.

THESE COUNTS DRIFT — DO NOT TURN THEM INTO EXPECTED RESULTS The counts above are the later of two sweeps run a few hours apart on 20 July 2026, and on some figures the two sweeps do not agree: view=software on KOI_PLTS read 1,044 in the earlier sweep and 1,046 a few hours later the same day, and the no-view inventory total on KOI_PLTS rose from 5,644 to 5,646 over the same interval. The drift is per tenant, not universal — KOI_PAET returned 3,447 in both sweeps. And it is between runs, never within one: each instance's unfiltered inventory was probed twice inside a single run and returned the same figure both times (attempt_within_run 1 and 2 in evidence/followup-probes.json). So this is inventory growth on the tenant that moved, not the API wobbling — and both readings are correct as of their own run. A test step must therefore assert a shape: HTTP 200, a total_count present and non-zero, the expected fields on the first record. A reader whose numbers differ from these has not failed a test.

Missing from the dropdown but valid: all_items, mcp_servers, repositories. Nothing in the YAML is invalid — the list is incomplete.

[LIVE] This matters most for mcp_servers, the MCP-server audit case: it returned HTTP 200 with total_count 21 on KOI_PAET and 42 on KOI_PLTS, but the argument dropdown will never offer it. repositories is likewise real data (15 / 77) that the dropdown hides.

```
!koi-inventory-list view=mcp_servers limit=50
```

A6. koi-inventory-search fails on its filter

Symptom	The command errors instead of returning items. What the error says depends on how the filter is wrong — and the three cases have different causes.
Cause	The API wants the query-builder object — a combinator and a rules array — and the pack documents the argument as "query builder syntax" without showing the shape. What the pack lacks is an example, not a usable error message.
Evidence	[LIVE] The structured filter below returned HTTP 200 with total_count 145 on KOI_PAET and 75 on KOI_PLTS. Two of the three failure modes below were reproduced as HTTP responses, on both instances: the filter key omitted from the request returned 500, and an empty filter object {} returned 400. [CODE] The third was not reproduced and cannot be — it is a code fact, read from Koi.py line 468, where the integration raises before any HTTP call is made. There is no response to observe, and it was never executed here. [YAML] filter_json and filter_raw_json_entry_id are both optional in the YAML. [CODE] The refusal when neither is given lives in the integration code, not in the YAML.
Do this	Always pass a complete {combinator, rules} object. Read the 400 body before guessing — it names the offending key. For anything long, put the JSON in a war-room file and pass filter_raw_json_entry_id, which takes priority over filter_json.

Three distinct failure modes. Do not conflate them — they fail in different places, and they are not all known in the same way:

What you do	Where it fails	How this is known	What you get
Call the command with neither filter_json nor filter_raw_json_entry_id	The integration, before any HTTP call	[CODE] read from Koi.py line 468 — never executed, no API call to observe	Koi.py raises "Either 'filter_json' or 'filter_raw_json_entry_id' must be provided." No API call is made at all.
Pass a filter that is present but malformed, e.g. {}	The API	[LIVE] reproduced on both instances	HTTP 400 — and the body names the problem: filter.combinator must be one of the following values: and, or ... filter.rules must be an array. The 400 is not bare and not unexplained.

What you do	Where it fails	How this is known	What you get
Omit the filter key from a direct API request	The API	[LIVE] reproduced on both instances	HTTP 500 Internal Server Error. Reachable only by calling the API yourself; the command cannot produce it, because the integration stops you first.

```
!koi-inventory-search filter_json={`{"combinator": "and", "rules": [{"field": "risk_level", "operator": "=", "value": "high"}]}`
```

[LIVE] That exact filter object is the verified working one — total_count 145 on KOI_PAET, 75 on KOI_PLTS. Those are the counts this filter returned; no comparison was run against koi-inventory-list with the same risk level, so nothing is claimed about the two agreeing. Only the operator "=" was tested — no other operator was tried, so none is asserted to be valid or invalid. [DERIVED] The war-room command line wrapping it is illustrative — the commands could not be executed through XSIAM on this tenant (section 3).

Arguments this command accepts, straight from the pack:

Argument	Required	Default	Description
filter_json	no	—	Advanced filter using query builder syntax as a JSON string. Either filter_json or filter_raw_json_entry_id must be provided.
filter_raw_json_entry_id	no	—	War Room entry ID of a JSON file containing the filter object. Takes priority over filter_json when both are provided.
page	no	—	Page number for pagination (1-based). When provided, fetches a single page and ignores the limit argument.
page_size	no	—	Number of results per page (default: 50, max: 500). Used in single-page mode with the page argument.
limit	no	50	Maximum total number of inventory items to return (default: 50, max: 1000). When provided without page, auto-paginates to collect up to this many items.
sort_by	no	first_seen	Column to sort by.
sort_direction	no	desc	Sort direction.

A7. Two instances, one dataset, no way to tell them apart

Symptom	Row counts look doubled, or an alert appears twice, and nothing in the row says which instance produced it.
Cause	Both instances have Fetch events enabled and both write Alerts and Audit into the same koi_koi_raw dataset. The pack ships no field identifying the producing instance.
Evidence	[LIVE] Both KOI_PAET and KOI_PLTS are configured identically apart from their API key: isFetchEvents true on both, event_types_to_fetch ["Alerts", "Audit"] on both. [CODE] The destination dataset is not configurable — Koi.py calls send_events_to_xsiam(vendor="koi", product="koi"), which is what makes it koi_koi_raw. That both instances land in that one dataset is therefore a consequence of their configuration and of the integration code, not something observed row by row: as the Cause row says, the pack ships no field identifying the producing instance, so no row can be attributed to an instance in the first place.

Do this	Do not try to separate them inside the dataset — there is no field to do it with. Separate them upstream: give each instance a distinct KOI tenant scope, or disable Fetch events on one instance if the two KOI tenants overlap.
----------------	---

The two KOI tenants behind these instances hold very different volumes, which is itself a useful discriminator when you are sanity-checking a count. Where a figure moved between the two sweeps of 20 July 2026 it is shown as earlier → later:

Measure (via API, 20 Jul 2026 — a snapshot; the figures that moved between the two runs carry an arrow)	KOI_PAET	KOI_PLTS
Inventory items	3,447	5,644 → 5,646
Policies	32	28
Allowlist entries	0 (empty)	5
Blocklist entries	0 (empty)	17
view=mcp_servers	21	42
view=software	406	1,044 → 1,046
Alerts available via API	296	48,526
Audit records available via API	8,789	96,196

[LIVE] Two things follow. These two tenants differ by two orders of magnitude on alerts, so any row-count assertion in a test plan must name the tenant it was measured on — as every figure in the table above does. And some of the counts drift between runs — view=software on KOI_PLTS read 1,044 in the earlier sweep and 1,046 a few hours later the same day, and the no-view inventory total on KOI_PLTS rose from 5,644 to 5,646 over the same few hours, while KOI_PAET returned 3,447 in both sweeps — so none of the figures above is a pass/fail threshold. Only a figure that moved carries an arrow; the reading to the right of it is the later one, and the A5 matrix prints that same later reading. Assert on shape instead: HTTP 200, a total_count present and non-zero.

A8. The instance runs through an engine — your reachability test is testing the wrong path

Symptom	curl or a network test from the tenant to api.prod.koi.security succeeds, and the commands still fail.
Cause	Both instances execute on an engine, not on the tenant. The tenant's own egress is not the path the commands take, so a test from the tenant proves nothing about it.
Evidence	[LIVE] Engine c3664d21-63fb-4b2c-b16c-56cd547a3d79, propagationLabels ["all"], on both instances.
Do this	Run the reachability test from the engine host, against the same base URL and with the same proxy settings the instance carries (insecure and proxy are both false on these instances). Test integration on the instance itself exercises the real path; ad-hoc tests from elsewhere do not.

THE GENERAL FORM OF THIS MISTAKE Any evidence gathered from a machine that is not the one running the integration is circumstantial. That applies to DNS, TLS interception, proxy configuration and egress filtering alike.

A9. Allowlist and blocklist "return nothing" — checking total_count

Symptom	A check written as "assert total_count > 0" fails on koi-allowlist-get and koi-blocklist-get even when the lists are fine.
Cause	Those two endpoints return only an items array. There is no total_count on them, unlike /policies and /inventory. Reading a field that never exists is not the same as an empty list.
Evidence	[LIVE] /policies/allowlist and /policies/blocklist return items only. Both lists were genuinely empty on KOI_PAET, and 5 / 17 on KOI_PLTS.
Do this	Assert on the length of the items array. An empty items array is a correct, non-error result — it means the list is empty, not that the call failed.

[YAML] Both commands take no arguments at all (koi-allowlist-get: 0 args, koi-blocklist-get: 0), so there is nothing to misconfigure on the call itself.

```
!koi-allowlist-get
!koi-blocklist-get
```

A10. Context from one inventory command disappears after the next one runs

Symptom	A playbook reads Koi.Inventory.item_id and gets the wrong item, or a value that was there a task ago is gone.
Cause	3 commands declare the identical Koi.Inventory item paths, and both policy commands declare the identical Koi.Policy.* paths. Same key, last writer wins.
Evidence	[YAML] 36 of 68 distinct context paths are declared by more than one command — 131 declarations over 68 paths, a difference of 63 redundant declarations. Not observed running here: the commands could not be executed through XSIAM on this tenant (section 3), so this follows from what the pack declares, not from a watched overwrite.
Do this	Read the context immediately after each call, or copy it into a task-specific key with Set before calling the next inventory command. Do not assume a prefix survives a second command.

Prefix	Commands that overwrite each other	Declares the prefix but collides with nothing
Koi.Inventory	koi-inventory-item-get koi-inventory-list koi-inventory-search	koi-inventory-item-endpoints-list (writes only Koi.Inventory.Endpoint.*)
Koi.Policy	koi-policy-list koi-policy-status-update	—

Three commands, not four. koi-inventory-item-endpoints-list writes only the 10 nested Koi.Inventory.Endpoint.* paths. It shares no context path with the other three, so it neither overwrites them nor is overwritten by them.

A11. Alert counts look inflated — the same alert appears hundreds of times

Symptom	A count() over Alerts is far larger than the number of real alerts. One alert shows up as hundreds of near-identical rows in koi_koi_raw, and every widget, query or triage step that counts alert rows — or fires once per row — is overstated.
Cause	The integration re-sends every still-open alert on each fetch cycle, so koi_koi_raw holds one row per alert PER FETCH, not one row per alert. With eventFetchInterval = 1 minute, an alert that stays open for hours is re-inserted once a minute. Audit records are point-in-time and are not duplicated.

Evidence	[LIVE] Measured 21 July 2026. Over the last 24 hours the Alerts stream held 734 rows for just 3 distinct alerts — a 244.7× inflation; over 90 days, 1,048 rows for 317 distinct alerts (3.3×). Audit is unaffected: 257 rows / 257 distinct over 24 h and 20,148 / 20,148 over 90 d (1.0×). Within one notification, 357 rows carried 1 distinct _time and 1 distinct message but 357 distinct _insert_time — the same alert re-inserted 357 times, once per fetch minute. Nothing dedupes on the way in, because the pack ships no parsing rule.
Do this	Dedupe every alert count on the notification identity. The only correct key is metadata.notification_event_id. On already-ingested rows there is no promoted column, so extract it inline with json_extract_scalar(metadata, "\$.notification_event_id"). Never count() alert rows directly.

Four candidate identifiers were measured over the same 90-day window. Only one identifies the alert occurrence:

Field	Distinct / 1,048 rows	What it identifies
_id	1,048	the row — counts every duplicate
metadata.notification_event_id	317	the alert occurrence — the correct key
observables[event.id] (koi_event_id)	20	the scan batch — far too coarse
finding_info.uid (finding_uid)	3	the finding / policy definition — far too coarse

Corrected count — dedupe first, then count:

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts"
| alter koi_notification_id = json_extract_scalar(metadata, "$.notification_event_id")
| comp count_distinct(koi_notification_id) as alerts
```

[LIVE] On this data (21 July 2026), count() over Alerts returns 734 where count_distinct(koi_notification_id) returns 3. [DERIVED] The query text above is composed from those verified field names and was not executed as written; metadata is a plain object, not an array, so no coalesce is needed.

EVERY count() OVER ALERTS IS OVERSTATED The integration re-sends every still-open alert on each 1-minute fetch, so a row count is multiplied by however many fetch cycles each alert survived. Any triage playbook that reacts per alert row fires repeatedly for one real alert. Dedupe on the notification id — count_distinct(koi_notification_id) — and on historical rows extract it inline: json_extract_scalar(metadata, "\$.notification_event_id"). Audit is unaffected; this is Alerts-only.

A12. A marketplace value from an event is rejected with HTTP 400

Symptom	A command that takes a marketplace argument fails with HTTP 400 when the marketplace value came from an event in koi_koi_raw. The value looks valid but the API refuses it.
Cause	The marketplace field in koi_koi_raw uses SHORT forms (software_windows, chrome, vsc, ...). The API and the pack's YAML predefined list use LONG forms (windows, chrome_web_store, vscode, ...). The two vocabularies differ for most values — including the single most common event value, software_windows (which the API only accepts as windows) — so a value read from an event and passed to a command unchanged is usually rejected. Some values do coincide and pass through: npm and pypi (pypi is the second most common event value overall), plus homebrew, chocolatey, cursor, docker and claude_desktop_extensions.

Evidence	[LIVE] Verified 21 July 2026 by testing every marketplace value against GET /inventory?marketplace=. The koi_koi_raw marketplace field uses short forms and the API / YAML predefined list uses long forms; the two coincide for only a handful of values and differ for the majority. In a controlled operator test the same day, 16 of 19 real events (84 %) carried a value that failed HTTP 400 unmapped and succeeded once mapped — only pypi worked untouched. [YAML] The 22 long-form names are the predefined list on the marketplace argument.
Do this	Map the event short form to the API long form between reading an event and calling a command. Three event values (built_in, ollama, side_loaded) have no API equivalent at all — treat them as "unknown marketplace" and do not pass them on.

[YAML] 7 commands take a marketplace argument and validate it against the 22 long-form names; koi-inventory-item-endpoints-list and koi-inventory-item-get require it. Passing a raw event value to any of them fails with HTTP 400.

Event short form → API long form. The mapping is the fact; the two count columns are a 21 Jul 2026 snapshot and drift:

Event value (koi_koi_raw)	Events	API / YAML value	API items	Note
software_windows	5,301	windows	214	must be mapped
pypi	4,674	pypi	1,990	matches — passes through unchanged
chrome	891	chrome_web_store	63	must be mapped
built_in	829	– none –	—	installation_method value leaking into the field
npm	775	npm	325	matches — passes through unchanged
software_mac	617	mac	192	must be mapped
homebrew	231	homebrew	322	matches — passes through unchanged
vsc	175	vscode	64	must be mapped
chocolatey	91	chocolatey	28	matches — passes through unchanged
cursor	88	cursor	11	matches — passes through unchanged
github	65	github_mcp_registry	0	must be mapped
edge	48	edge_add_ons	11	must be mapped
firefox	19	firefox_add_ons	22	must be mapped
docker	15	docker	5	matches — passes through unchanged
npp	12	notepad++	5	must be mapped
openvsx	10	open_vsx_registry	0	must be mapped
jet	5	jetbrains	1	must be mapped
ollama	5	– none –	—	not in the API list

Event value (koi_koi_raw)	Events	API / YAML value	API items	Note
claudio_desktop_extensions	5	claudio_desktop_extensions	3	matches — passes through unchanged
side_loaded	1	— none —	—	installation_method value leaking into the field

[LIVE] Where the Event and API columns differ, the value is rejected with HTTP 400 if passed to a command unchanged; where they match (Note column), it passes through. The highest-frequency event value, software_windows, differs, so most real traffic needs the mapping (pypi, the second most common, is one of the few that passes through). built_in, ollama, side_loaded are not marketplaces at all (built_in and side_loaded are installation_method values that leak into the field; ollama is simply absent from the API's list), so they must be dropped rather than mapped.

[LIVE] In the companion pack (separate content — see the cover), KOI Ext - Extract Alert Context reads item.marketplace straight from the alert payload and downstream playbooks pass it to koi-inventory-item-get, koi-inventory-item-endpoints-list and koi-blocklist-items-add. The mapping above must be applied between reading the event and calling the command.

MAP EVENT VALUES BEFORE CALLING A COMMAND Do not feed a marketplace value from an event straight into a command unless you have confirmed it is one of the few that coincide. Map the short form to the long form first. The chrome rows fail HTTP 400 unmapped, and any audit-driven flow using software_windows — the most common value in the dataset — fails likewise. Treat built_in, ollama, side_loaded as unknown and drop them.

A13. A host stops producing events — KOI is run-on-demand, and nothing scanned it

Symptom	koi_koi_raw is populated overall, but one host has produced no events for days. A tester watching a single quiet host sees an empty result and reports collection as broken.
Cause	KOI has no resident agent on Windows — it is run-on-demand. Between scans nothing runs on the host, so nothing is generated. A scan must be triggered before any events appear, and KOI is change-driven: a scan of an unchanged host still produces nothing.
Evidence	[LIVE] Re-verified first-hand 21 July 2026. Two test hosts (win-workstation, koi-win-test) had produced no events since 15 July — six days — while other hosts reported continuously; nothing was wrong with them, nothing had run the scan. Running KOI Deployment Script - Windows via core-script-run returned COMPLETED_SUCCESSFULLY in 135 s, and a scan of an unchanged host produced no events at all. [INHERITED] The earlier custom-pack investigation established the same run-on-demand behaviour (Part B, B1).
Do this	Do not read an empty result on one host as a collection failure. Confirm a scan has actually run — trigger KOI Deployment Script - Windows through core-script-run — and confirm something changed on the host since the last scan. After a successful scan, events are queryable in koi_koi_raw within roughly 4–10 minutes.

SCAN, THEN CHANGE, THEN WAIT Three things must all be true before a host produces events: (1) a scan has run — KOI is run-on-demand, there is no resident agent between scans; (2) something changed since the last scan — a re-scan of an unchanged host is a no-op, and a re-install of an already-inventoried item is also a no-op because KOI is change-driven; (3) a few minutes have passed — ingestion takes roughly 4–10 minutes after the scan completes. An empty dataset on a quiet host is expected, not a fault.

[LIVE] The scan is driven the same way the Script Runner workflow drives it — a core-script-run of KOI Deployment Script - Windows (name → uid → run). **[PACK]** That Script Runner workflow is additional content

in the separate KOI Content Extension pack, not part of the Marketplace pack; core-script-run itself is Cortex-native.

A14. You installed something but KOI doesn't see it

Symptom	A package, extension or repository was installed on a scanned host, a scan ran, and it still does not appear in KOI's inventory or events.
Cause	KOI scans user-profile locations and ignores the SYSTEM profile. If the install landed in a SYSTEM-profile path — as anything the Cortex EDR agent installs does, because the agent runs as SYSTEM — KOI never sees it. It is the PATH that decides, not the identity of the installing process.
Evidence	[LIVE] Verified 21 July 2026 by a controlled three-way comparison, all three installs driven by a SYSTEM process. <code>tabulate==0.9.0</code> landed in the SYSTEM profile and was NOT detected — <code>/inventory/tabulate/endpoints?version=0.9.0</code> returned HTTP 404. <code>inflection==0.5.1</code> in a user profile WAS detected (event 09:55:12), and a git clone into a user profile WAS detected (event 09:48:06). Same installing identity throughout, so the path — not the process — is the variable.
Do this	Install into a user-profile path if you want KOI to inventory it. For detection testing, make the change in a user profile the agent scans, and use an item verified absent from the host beforehand — KOI is change-driven, so a re-install of an already-inventoried item produces nothing.

Package / item	Target path	Detected?
<code>tabulate==0.9.0</code>	<code>C:\Windows\system32\config\systemprofile\...\site-packages</code> (SYSTEM profile)	No — 404
<code>inflection==0.5.1</code>	<code>C:\Users\amahmoud\AppData\Roaming\Python\...</code> (user profile)	Yes — 09:55:12
<code>octocat/Hello-World</code> (git clone)	<code>C:\Users\amahmoud\Documents\koi-test-repo</code> (user profile)	Yes — 09:48:06

PATH, NOT PROCESS KOI scans user-profile locations only. A SYSTEM-profile install is invisible no matter what installed it — and anything the Cortex EDR agent runs installs as SYSTEM. This is also why detection tests can be automated through the agent: the requirement is a user-profile path, not an interactive user. B6 is the same surprise from the opposite direction — KOI's own bundled Python makes PyPI packages appear on hosts where nobody installed Python.

3. Environment Finding — Automating Command Tests on This Tenant

[LIVE] This is a tenant limitation, not a pack fault. It is recorded because anyone trying to automate a command sweep will hit it, lose a day to it, and conclude the pack is broken.

What was attempted	What happened
POST /investigations/search	303 redirect to /#/404. demisto-sdk run depends on this endpoint, so it cannot run commands on this tenant at all.
POST /incident	HTTP 200 with an empty body. No incident is created — XSIAM does not accept incident creation over the XSOAR API.
Running a command in the playground	An API-key user's playground is auto-created as a malformed stub (type: 0, modified 0001-01-01T00:00:00Z). Every command run there — including the built-in !Print — returns Panic [runtime error: invalid memory address or nil pointer dereference] (56).

WHAT THIS DOES AND DOES NOT ESTABLISH Command behaviour was therefore checked one layer down, against the same endpoints, with the same bearer auth and the same API keys the two instances use — for 8 of the 13 commands. The other 5 change tenant state and were deliberately not run at all. For those 8, this establishes that the endpoints exist, that parameters are accepted or rejected as described, and what the response shapes are. It establishes NOTHING about the XSOAR side: no command was ever executed in a war room on this tenant, so no human-readable output, no war-room table and no context mapping was observed. Those are asserted from the YAML and from Koi.py — two different artefacts, tagged [YAML] and [CODE] respectively wherever used — and must not be read as tested.

[LIVE] One authentication fact did come out of this: an invalid bearer token returns HTTP 401 {"message":"Unauthorized","statusCode":401}. [UNVERIFIED] No 403 was ever observed from this machine. Do not read a 403 as proof of blocked egress — that mapping was never seen here.

To close the gap: run the command sweep from the XSIAM UI war room, which does not use the API path that fails, and compare the results against evidence/command-sweep.json in this repository.

[UNVERIFIED] No claim is made anywhere in this guide about a command's war-room table layout or its human-readable output.

Part B — Endpoint Findings Inherited from the Custom-Pack Investigation

INHERITED — NOT RE-VERIFIED IN THIS SESSION Everything in Part B was established during the earlier custom-pack investigation on a live Windows endpoint. These findings are pack-independent — they concern the KOI agent on the endpoint, not the content pack — which is why they are reproduced here. They were NOT re-verified against this Marketplace pack or in this session. Treat them as strong prior evidence, not as fresh observation. Every heading in this part is blue for that reason. This part adds nothing to the inherited notes: where a practical next step goes beyond what those notes actually record, it is tagged [UNVERIFIED] and marked untested rather than presented as a finding.

B1. There is no resident KOI agent on Windows

[INHERITED] KOI on Windows is run-on-demand: there is no service, no resident process and no scheduled task between runs. A scheduled Job is therefore the scan scheduler.

Consequence: no Job run means no scan — not a delayed scan, not a partial one. An investigation into stale endpoint data starts with the Job, not the endpoint.

[PACK] Note for this pack specifically: the Job-plus-Script-Runner workflow is content from the custom pack. It is not part of the Marketplace pack, which ships an integration only. That is a statement about what the two pack directories contain, established by inspecting them — not an observation of this tenant or its API, and not something the integration YAML says, which is why it is tagged [PACK] rather than [LIVE] or [YAML].

B2. The file map — the four files recorded under C:\ProgramData\Koi

[INHERITED] The inherited notes record these four files under **C:\ProgramData\Koi**. Their descriptions below go no further than those notes; nothing about install layout elsewhere on the disk was recorded, so nothing is claimed about it here.

File	What the inherited notes say
settings.json	Scan configuration; pulled from the backend rather than written locally by the scan (see B3)
agent_policies.json	Enforcement policy set; carries the cid (KOI tenant id) and the deviceId for this endpoint
agent_activity.jsonl	Agent activity record. Named in the inherited file list; its contents were not characterised, and no expected size is claimed
agent_enforcement.log	Enforcement engine log. Named in the inherited file list; its contents were not characterised, and no expected size is claimed

B3. Only a fresh mtime proves an authenticated round trip

[INHERITED] settings.json and agent_policies.json are not written locally by the scan — they are pulled from the backend after the agent authenticates. A new modification time on those two files is the only evidence that the complete round trip succeeded. Reachability alone proves nothing.

```
dir C:\ProgramData\Koi
```

[UNVERIFIED] What follows is a suggested reading of that check, not a tested one. The inherited notes establish only the sentence above — that a fresh mtime is the proof of a round trip. The three interpretations below have not been reproduced in this session or in the notes, and are offered as a starting point for investigation, not as findings.

Observation	Suggested next step — untested
Backend responds, config files fresh	Consistent with a healthy round trip including credentials. Look elsewhere
Backend responds, config files stale	Network is reachable, so look at enrolment, credentials, or a run failing before its config fetch
Backend does not respond	Look at the network path: egress filtering, proxy, TLS interception or DNS

B4. Extension display names come from _locales, not the manifest

[INHERITED] Chrome stores a localization placeholder in manifest.json and the real display name in _locales\<lang>\messages.json. KOI resolves it the way Chrome does.

```
C:\Users\<user>\AppData\Local\Google\Chrome\User
Data\Default\Extensions\<id>\<version>_0\manifest.json
```

So a manifest whose name field reads __MSG_<key>__ is not corruption. Comparing KOI's display name against manifest.json:name alone produces a false mismatch.

B5. Installed software needs both registry hives

[INHERITED] Querying only the first hive misses every 32-bit application on a 64-bit host.

```
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*
HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*
```

B6. KOI bundles its own Python

[INHERITED] KOI ships a bundled Python runtime, so PyPI packages appear in inventory on hosts where nobody ever installed Python — KOI is partly inventorying its own interpreter. Do not treat that as a false positive or as shadow IT.

B7. The powershell -Command - trap

[INHERITED] powershell -Command - executes piped stdin line by line, so multi-line foreach and if blocks break apart. You get a header, no rows, and no error — a silently wrong answer.

Use -EncodedCommand with base64 UTF-16LE instead:

```
python3 -c "import base64;print(base64.b64encode(open('s.ps1').read().encode('utf-16-le')).decode())"
powershell -NoProfile -EncodedCommand <base64>
```

B8. Install path decides event collection

[INHERITED] A pack zip built for the wrong marketplace target carries isfetchevents: false and ships no rules. This is the root cause behind symptom A1 and the reason A1 tells you to reinstall rather than edit the instance.

4. Fast Triage Order

Work down the list. The first check that fails localises the fault.

1. **Is this the right pack?** Count the commands — this pack has exactly 13 (section 1.1). Anything device-centric means you are on the custom pack and this guide does not apply.
2. **Commands or events?** They travel different routes. Failing commands point at the instance, the engine or the API; a missing dataset points at collection (A1); a single quiet host that has simply not been scanned produces nothing even while collection is healthy (A13).
3. **Does the instance have Fetch events at all?** No toggle means the wrong pack build — reinstall (A1, B8).
4. **Does koi_koi_raw resolve, and does it split into Audit and Alerts?** If it resolves but your query returns nothing, filter on source_log_type first (section 2.2).
5. **Are you counting Alerts?** The Alerts stream is massively duplicated — one row per alert per fetch. Dedupe on the notification id before counting, or before firing per row (A11).
6. **Is the query reading a field that is never populated?** xdm.* never fills (A2); alert_type is always null (A4); resources/observables/metadata need json_extract (A3).
7. **Is the command failing with HTTP 400?** Check view against the API's accepted set, not the dropdown (A5), check that filter_json carries {combinator, rules} (A6), and if the 400 is on a marketplace value, confirm it is the API long form and not the event short form (A12).
8. **Installed something the scan should have caught?** It is inventoried only if it is in a user-profile path — a SYSTEM-profile install is invisible (A14).
9. **Are you testing the path the integration actually uses?** These instances run on an engine — a test from the tenant proves nothing (A8).
10. **Only then look at the endpoint,** and read Part B as prior evidence rather than as verified fact for this pack.

5. Command Reference — Generated From the Pack

[YAML] Every row below is emitted at build time from Packs/Koi/Integrations/Koi/Koi.yml (md5 5497cdddedeb0c0d7d0b371aa075a64c). Nothing here is hand-typed, so it cannot drift from the pack.

6 kinds of badge appear on the command blocks below. Every block carries exactly one of the first two; the other 4 appear only where they apply, so a block may carry one badge or several.

[LIVE] "API exercised read-only, 20 Jul 2026" marks the 8 commands whose endpoint was called on that date. "NOT RUN — changes tenant state" marks the 5 that were deliberately never called. No command in this section was executed through XSIAM (section 3), so nothing here describes war-room output. [YAML] The remaining 4 come from the pack: "execution: true (potentially harmful)" on the 2 commands the pack flags; "changes tenant state, NOT flagged execution: true" on the 3 that mutate state without that flag; "no context outputs — cannot be branched on" on the 4 that declare no outputs; and "NOT safely repeatable — see the description above" on koi-get-events.

[YAML] That last badge rests on two different sources. The pack's own description of koi-get-events says to use it for development and debugging only, as it may produce duplicate events, exceed API rate limits or disrupt the fetch mechanism, and the YAML declares a `should_push_events` argument (default false) described as pushing events to Cortex XSIAM. [CODE] That the destination is `koi_koi_raw` is not in the YAML — it follows from `send_events_to_xsiam(vendor="koi", product="koi")` in `Koi.py`, read from the source. The argument is handled by the integration and is not a parameter of the underlying GET, so nothing was pushed by the direct API requests made here.

Being non-mutating is not the same as being repeatable: 7 of the 8 commands whose API was exercised are freely repeatable, and that one is not.

koi-allowlist-get

Retrieves all items in the allowlist.

GET /policies/allowlist · 0 arguments · 9 outputs · API exercised read-only, 20 Jul 2026

No arguments.

Context outputs:

Context path	Type	Description
<code>Koi.Allowlist.item_id</code>	String	The unique identifier of the allowlist item.
<code>Koi.Allowlist.item_name</code>	String	The name of the allowlist item.
<code>Koi.Allowlist.item_display_name</code>	String	The display name of the allowlist item.
<code>Koi.Allowlist.marketplace</code>	String	The marketplace of the allowlist item (e.g., vscode).
<code>Koi.Allowlist.publisher_name</code>	String	The publisher name of the allowlist item.
<code>Koi.Allowlist.package_name</code>	String	The package name of the allowlist item.
<code>Koi.Allowlist.notes</code>	String	Notes associated with the allowlist item.
<code>Koi.Allowlist.created_by</code>	String	The user who created the allowlist item.
<code>Koi.Allowlist.created_at</code>	Date	The creation time of the allowlist item in ISO 8601 format.

koi-allowlist-items-add

Adds one or more items to the global allowlist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk addition from a JSON file.

POST /policies/allowlist · 5 arguments · 0 outputs · **changes tenant state, NOT flagged execution: true** · **no context outputs — cannot be branched on** · **NOT RUN — changes tenant state**

Argument	Req.	Default	Predefined values / notes
item_id	no	—	free text
marketplace	no	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
created_by	no	—	free text
notes	no	—	free text
items_list_raw_json_entry_id	no	—	free text

koi-allowlist-items-remove

Removes one or more items from the global allowlist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk removal from a JSON file.

DELETE /policies/allowlist · 5 arguments · 0 outputs · **execution: true (potentially harmful)** · **no context outputs — cannot be branched on** · **NOT RUN — changes tenant state**

Argument	Req.	Default	Predefined values / notes
item_id	no	—	free text
marketplace	no	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
created_by	no	—	free text
notes	no	—	free text
items_list_raw_json_entry_id	no	—	free text

koi-blocklist-get

Retrieves all items in the blocklist.

GET /policies/blocklist · 0 arguments · 9 outputs · **API exercised read-only, 20 Jul 2026**

No arguments.

Context outputs:

Context path	Type	Description
Koi.Blocklist.item_id	String	The unique identifier of the blocklist item.
Koi.Blocklist.item_name	String	The name of the blocklist item.
Koi.Blocklist.item_display_name	String	The display name of the blocklist item.
Koi.Blocklist.marketplace	String	The marketplace of the blocklist item (e.g., vscode).
Koi.Blocklist.publisher_name	String	The publisher name of the blocklist item.
Koi.Blocklist.package_name	String	The package name of the blocklist item.
Koi.Blocklist.notes	String	Notes associated with the blocklist item.
Koi.Blocklist.created_by	String	The user who created the blocklist item.
Koi.Blocklist.created_at	Date	The creation time of the blocklist item in ISO 8601 format.

koi-blocklist-items-add

Adds one or more items to the global blocklist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk addition from a JSON file.

POST /policies/blocklist · 5 arguments · 0 outputs · **changes tenant state, NOT flagged execution: true** · **no context outputs — cannot be branched on** · **NOT RUN — changes tenant state**

Argument	Req.	Default	Predefined values / notes
item_id	no	—	free text
marketplace	no	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
created_by	no	—	free text
notes	no	—	free text
items_list_raw_json_entry_id	no	—	free text

koi-blocklist-items-remove

Removes one or more items from the global blocklist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk removal from a JSON file.

DELETE /policies/blocklist · 5 arguments · 0 outputs · **execution: true (potentially harmful)** · **no context outputs — cannot be branched on** · **NOT RUN — changes tenant state**

Argument	Req.	Default	Predefined values / notes
item_id	no	—	free text
marketplace	no	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
created_by	no	—	free text
notes	no	—	free text
items_list_raw_json_entry_id	no	—	free text

koi-get-events

Gets events from KOI. Use this command for development and debugging only, as it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism.

GET /alerts and /audit-logs · 5 arguments · 4 outputs · **API exercised read-only, 20 Jul 2026** · **NOT safely repeatable — see the description above**

Argument	Req.	Default	Predefined values / notes
event_type	no	Alerts,Audit	Alerts, Audit
limit	no	50	free text
start_time	no	—	free text
end_time	no	—	free text
should_push_events	no	false	true, false

Context outputs:

Context path	Type	Description
KOI.Event.id	String	The unique identifier of the event.
KOI.Event.source_log_type	String	The source log type of the event (Alerts or Audit).
KOI.Event._time	Date	The timestamp of the event in ISO 8601 format.
KOI.Event.created_at	Date	The creation time of the event (audit logs).

koi-inventory-item-endpoints-list

Retrieves a paginated list of endpoints that have a specific item installed.

GET /inventory/{item_id}/endpoints · 6 arguments · 10 outputs · [API exercised read-only, 20 Jul 2026](#)

Argument	Req.	Default	Predefined values / notes
item_id	yes	—	free text
marketplace	yes	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
version	yes	—	free text
page	no	—	free text
page_size	no	—	free text
limit	no	50	free text

Context outputs:

Context path	Type	Description
Koi.Inventory.Endpoint.id	String	The unique identifier of the endpoint device.
Koi.Inventory.Endpoint.hostname	String	The hostname of the endpoint.
Koi.Inventory.Endpoint.os	String	The operating system of the endpoint.
Koi.Inventory.Endpoint.platform	String	The platform where the item is installed on this endpoint.
Koi.Inventory.Endpoint.serial	String	The serial number of the endpoint device.
Koi.Inventory.Endpoint.last_logged_on_user	String	The last logged on user of the endpoint.
Koi.Inventory.Endpoint.activation_status	String	The activation status of the endpoint.
Koi.Inventory.Endpoint.path	String	The installation path of the item on the endpoint.
Koi.Inventory.Endpoint.first_seen	Date	The date the item was first seen on this endpoint in ISO 8601 format.
Koi.Inventory.Endpoint.last_seen	Date	The date the item was last seen on this endpoint in ISO 8601 format.

koi-inventory-item-get

Retrieves comprehensive details for a specific software item, extension, or package using its unique identifier, marketplace, and version.

GET /inventory/{item_id} · 3 arguments · 27 outputs · [API exercised read-only, 20 Jul 2026](#)

Argument	Req.	Default	Predefined values / notes
item_id	yes	—	free text
marketplace	yes	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
version	yes	—	free text

Context outputs:

Context path	Type	Description
Koi.Inventory.item_id	String	The unique identifier of the inventory item.
Koi.Inventory.item_display_name	String	The display name of the inventory item.
Koi.Inventory.marketplace	String	The marketplace source of the item.
Koi.Inventory.platforms	Unknown	List of platforms where the item is installed.
Koi.Inventory.publisher_name	String	The publisher name of the item.
Koi.Inventory.risk	Number	The numeric risk score of the item.
Koi.Inventory.risk_level	String	The risk level classification of the item.
Koi.Inventory.version	String	The version of the item.
Koi.Inventory.status	String	The governance status of the item.
Koi.Inventory.endpoint_count	Number	The number of endpoints where the item is installed.
Koi.Inventory.installs_count	Number	The total number of installs for the item.
Koi.Inventory.installation_method	String	The method used to install the item.
Koi.Inventory.is_first_party	Boolean	Whether the item is a first-party item.
Koi.Inventory.is_signed	Boolean	Whether the item is signed.
Koi.Inventory.first_seen	Date	The date the item was first seen in ISO 8601 format.
Koi.Inventory.last_seen	Date	The date the item was last seen in ISO 8601 format.
Koi.Inventory.last_used	Date	The date the item was last used in ISO 8601 format.

Context path	Type	Description
Koi.Inventory.released_at	Date	The release date of the item. Format: YYYY-MM-DD (e.g., 2023-01-15).
Koi.Inventory.short_description	String	A short description of the item.
Koi.Inventory.categories	Unknown	List of categories the item belongs to.
Koi.Inventory.findings	Unknown	List of findings associated with the item including severity and evidence.
Koi.Inventory.governed_details	Unknown	Governance policy details for the item.
Koi.Inventory.brew_category_koi	String	The Homebrew package category (Koi classification).
Koi.Inventory.browser_category_koi	String	The browser extension category (Koi classification).
Koi.Inventory.chocolatey_category_koi	String	The Chocolatey package category (Koi classification).
Koi.Inventory.ide_category_koi	String	The IDE extension category (Koi classification).
Koi.Inventory.software_category_koi	String	The software category (Koi classification).

koi-inventory-list

Retrieves a paginated list of items installed across your organization's endpoints. Supports extensive filtering by marketplace, platform, risk level, publisher, and specific categories.

GET /inventory · 21 arguments · 27 outputs · [API exercised read-only, 20 Jul 2026](#)

Argument	Req.	Default	Predefined values / notes
page	no	—	free text
page_size	no	—	free text
limit	no	50	free text
brew_category_koi	no	—	free text
browser_category_koi	no	—	free text
chocolatey_category_koi	no	—	free text
device_id	no	—	free text
finding_id	no	—	free text
first_seen	no	—	free text
ide_category_koi	no	—	free text
installation_method	no	—	marketplace, manual, built_in, side_loaded
item_display_name	no	—	free text
item_id	no	—	free text

Argument	Req.	Default	Predefined values / notes
marketplace	no	–	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf
platform	no	–	antigravity, aqua, arc, brave, brew, chatgpt_atlas, chocolatey, chrome, chromium, claude, clion, codex, comet, cursor, datagrip, dataspell, dia, edge, excel, firefox, fleet, golang, hugging_face, intellij_community, intellij, kiro, mac, npm, notepad++, opera, outlook, phpstorm, powerpoint, prisma_access_browser, pycharm, pypi, rider, rubymine, rustrover, vscode, webstorm, windsurf, word, windows, writerside
publisher_name	no	–	free text
risk_level	no	–	low, medium, high, critical, pending
software_category_koi	no	–	free text
sort_by	no	first_seen	first_seen, last_seen, item_display_name, item_id, version, marketplace, endpoint_count, risk, risk_level, status, installs_count, released_at, publisher_name
sort_direction	no	–	asc, desc
view	no	–	agentic_ai, ai_models, code_packages, extensions, os_packages, software

Context outputs:

Context path	Type	Description
Koi.Inventory.item_id	String	The unique identifier of the inventory item.
Koi.Inventory.item_display_name	String	The display name of the inventory item.
Koi.Inventory.marketplace	String	The marketplace source of the item.
Koi.Inventory.platforms	Unknown	List of platforms where the item is installed.
Koi.Inventory.publisher_name	String	The publisher name of the item.
Koi.Inventory.risk	Number	The numeric risk score of the item.

Context path	Type	Description
Koi.Inventory.risk_level	String	The risk level classification of the item.
Koi.Inventory.version	String	The version of the item.
Koi.Inventory.status	String	The governance status of the item.
Koi.Inventory.endpoint_count	Number	The number of endpoints where the item is installed.
Koi.Inventory.installs_count	Number	The total number of installs for the item.
Koi.Inventory.first_seen	Date	The date the item was first seen in ISO 8601 format.
Koi.Inventory.last_seen	Date	The date the item was last seen in ISO 8601 format.
Koi.Inventory.last_used	Date	The date the item was last used in ISO 8601 format.
Koi.Inventory.installation_method	String	The method used to install the item.
Koi.Inventory.short_description	String	A short description of the item.
Koi.Inventory.is_first_party	Boolean	Whether the item is a first-party item.
Koi.Inventory.is_signed	Boolean	Whether the item is signed.
Koi.Inventory.categories	Unknown	List of categories the item belongs to.
Koi.Inventory.findings	Unknown	List of findings associated with the item.
Koi.Inventory.governed_details	Unknown	Governance policy details for the item.
Koi.Inventory.released_at	Date	The release date of the item. Format: YYYY-MM-DD (e.g., 2023-01-15).
Koi.Inventory.brew_category_koi	String	The Homebrew package category (Koi classification).
Koi.Inventory.browser_category_koi	String	The browser extension category (Koi classification).
Koi.Inventory.chocolatey_category_koi	String	The Chocolatey package category (Koi classification).
Koi.Inventory.ide_category_koi	String	The IDE extension category (Koi classification).
Koi.Inventory.software_category_koi	String	The software category (Koi classification).

koi-inventory-search

Searches inventory items using advanced query builder filters. Provide a filter via 'filter_json' (inline JSON string) or 'filter_raw_json_entry_id' (War Room file entry ID). At least one filter source must be provided.

POST /inventory/search · 7 arguments · 27 outputs · [API exercised read-only, 20 Jul 2026](#)

Argument	Req.	Default	Predefined values / notes
filter_json	no	—	free text
filter_raw_json_entry_id	no	—	free text
page	no	—	free text
page_size	no	—	free text

Argument	Req.	Default	Predefined values / notes
limit	no	50	free text
sort_by	no	first_seen	first_seen, last_seen, item_display_name, item_id, version, marketplace, endpoint_count, risk, risk_level, status, installs_count, released_at, publisher_name
sort_direction	no	desc	asc, desc

Context outputs:

Context path	Type	Description
Koi.Inventory.item_id	String	The unique identifier of the inventory item.
Koi.Inventory.item_display_name	String	The display name of the inventory item.
Koi.Inventory.marketplace	String	The marketplace source of the item.
Koi.Inventory.platforms	Unknown	List of platforms where the item is installed.
Koi.Inventory.publisher_name	String	The publisher name of the item.
Koi.Inventory.risk	Number	The numeric risk score of the item.
Koi.Inventory.risk_level	String	The risk level classification of the item.
Koi.Inventory.version	String	The version of the item.
Koi.Inventory.status	String	The governance status of the item.
Koi.Inventory.endpoint_count	Number	The number of endpoints where the item is installed.
Koi.Inventory.installs_count	Number	The total number of installs for the item.
Koi.Inventory.first_seen	Date	The date the item was first seen in ISO 8601 format.
Koi.Inventory.last_seen	Date	The date the item was last seen in ISO 8601 format.
Koi.Inventory.last_used	Date	The date the item was last used in ISO 8601 format.
Koi.Inventory.installation_method	String	The method used to install the item.
Koi.Inventory.short_description	String	A short description of the item.
Koi.Inventory.is_first_party	Boolean	Whether the item is a first-party item.
Koi.Inventory.is_signed	Boolean	Whether the item is signed.
Koi.Inventory.categories	Unknown	List of categories the item belongs to.
Koi.Inventory.findings	Unknown	List of findings associated with the item.
Koi.Inventory.governed_details	Unknown	Governance policy details for the item.
Koi.Inventory.released_at	Date	The release date of the item. Format: YYYY-MM-DD (e.g., 2023-01-15).
Koi.Inventory.brew_category_koi	String	The Homebrew package category (Koi classification).

Context path	Type	Description
Koi.Inventory.browser_category_koi	String	The browser extension category (Koi classification).
Koi.Inventory.chocolatey_category_koi	String	The Chocolatey package category (Koi classification).
Koi.Inventory.ide_category_koi	String	The IDE extension category (Koi classification).
Koi.Inventory.software_category_koi	String	The software category (Koi classification).

koi-policy-list

Retrieves a list of all policies. Use 'page' and 'page_size' to fetch a specific page, or use 'limit' to auto-paginate and collect up to the specified number of policies. If 'page' is provided, 'limit' is ignored.

GET /policies · 3 arguments · 9 outputs · [API exercised read-only, 20 Jul 2026](#)

Argument	Req.	Default	Predefined values / notes
page	no	—	free text
page_size	no	—	free text
limit	no	50	free text

Context outputs:

Context path	Type	Description
Koi.Policy.id	Number	The unique identifier of the policy.
Koi.Policy.name	String	The name of the policy.
Koi.Policy.description	String	The description of the policy.
Koi.Policy.action	String	The action taken by the policy (e.g., block).
Koi.Policy.enabled	Boolean	Whether the policy is enabled.
Koi.Policy.group_ids	Unknown	List of group IDs associated with the policy.
Koi.Policy.creator_fullname	String	The full name of the policy creator.
Koi.Policy.created_at	Date	The creation time of the policy in ISO 8601 format.
Koi.Policy.updated_at	Date	The last update time of the policy in ISO 8601 format.

koi-policy-status-update

Enables or disables a policy by ID.

PUT /policies/{id} · 2 arguments · 9 outputs · **changes tenant state, NOT flagged execution: true** · **NOT RUN**
— changes tenant state

Argument	Req.	Default	Predefined values / notes
policy_id	yes	—	free text
enabled	yes	—	true, false

Context outputs:

Context path	Type	Description
Koi.Policy.id	Number	The unique identifier of the policy.
Koi.Policy.name	String	The name of the policy.
Koi.Policy.description	String	The description of the policy.
Koi.Policy.action	String	The action taken by the policy (e.g., block).
Koi.Policy.enabled	Boolean	Whether the policy is enabled.
Koi.Policy.group_ids	Unknown	List of group IDs associated with the policy.
Koi.Policy.creator_fullname	String	The full name of the policy creator.
Koi.Policy.created_at	Date	The creation time of the policy in ISO 8601 format.
Koi.Policy.updated_at	Date	The last update time of the policy in ISO 8601 format.

Appendix — Provenance of This Document

This guide is generated. Re-running the generator against the same four inputs produces the same document.

Input	Role
reference/marketplace-pack.json	[YAML] Mechanical extraction of the pinned Koi.yml. Every command, argument, default, predefined list, context path, configuration parameter and count in this document is read from it at build time.
VERIFIED_FACTS.md	[LIVE] / [CODE] / [PACK] / [INHERITED] Everything observed on the tenant and the KOI API on 20–21 July 2026 — including the 21 July end-to-end scan, marketplace-vocabulary and Alerts-duplication findings (\$7b–\$7e) folded into A11–A14 — the facts read from Koi.py and command_examples.txt, what inspection of the pack directory establishes, plus the \$8 endpoint findings inherited from the custom-pack investigation.
Packs/KoiContentExtension/	[PACK] The separate companion pack. Its name, version and content counts on the cover (parsing rules, modeling rules, playbooks, dashboard) are read from its pack_metadata.json and directory at build time. It is additional content, not part of the Marketplace pack.
evidence/followup-probes.json	[LIVE] Raw per-instance probe results, the LATER of the two sweeps. The view matrix in A5, the inventory and view rows in A7 (paired with the earlier sweep wherever the count moved) and the search failure-mode statuses in A6 are read from this file at build time, not transcribed. Its unfiltered_inventory section holds the later no-view inventory total for each instance, probed twice inside the one run — the generator fails rather than print a figure if those two attempts disagree.
evidence/command-sweep.json	[LIVE] The EARLIER sweep. Read so that counts which moved between the two runs are printed as earlier → later pairs, both readings coming from the files themselves rather than one being chosen silently over the other. A count that reads the same in both files is printed once, with no arrow.
docs/build_troubleshooting.js	This generator.

COUNTS IN THIS DOCUMENT ARE A SNAPSHOT On some figures the two sweeps of 20 July 2026 disagree, because inventory grew between them: view=software on KOI_PLTS read 1,044 in the earlier sweep and 1,046 a few hours later the same day, and the no-view inventory total on KOI_PLTS rose from 5,644 to 5,646. The drift is per tenant — KOI_PAET returned 3,447 in both sweeps — and between runs, not within one. Both readings are correct as of their own run, and both are read from the evidence files. Every count here is therefore a snapshot, never a pass/fail threshold — assert on shape instead.

Composed rather than quoted:

- [DERIVED] The XQL query strings in A1, A2, A3, A4 and A11. The dataset name, the discriminator field and every field name in them are [LIVE]; the query text itself was composed here and was not executed as written.
- [DERIVED] The war-room command lines (!koi...) shown as examples. Argument names and values are [YAML] or [LIVE]; the commands could not be executed through XSIAM on this tenant — see section 3.
- [DERIVED] The two ASCII path diagrams in section 2, which restate the [LIVE] engine and endpoint facts in a different form.

How tagging is applied — the method, not a guarantee:

- Tags are placed by hand, sentence by sentence. This document does not claim that every factual sentence in it carries a tag, and some do not: read an untagged sentence as inheriting the evidence level of the tagged statement it sits under, not as a stronger claim.
- Where tags are placed: the Evidence row of every Part A block, the lead statement of every Part B finding, and sentences elsewhere that assert something about the pack, the API or the tenant that a reader might otherwise have to take on trust.
- Deliberately untagged: the Symptom, Cause and Do this rows of Part A, the triage list in section 4, callouts, and headings. These are advice composed from the tagged Evidence row of the same block — not independent claims, and they carry no evidence of their own.
- Also untagged: consequence sentences that restate the operational implication of the tagged line immediately above them — for example B1's "no Job run means no scan" and B4's "a manifest whose name field reads __MSG_<key>__ is not corruption". They are reasoning from the [INHERITED] finding above them and are no better evidenced than it is.
- If any sentence, tagged or not, says more than its source supports, that is a defect in this generator; report it rather than trusting it.

Three claims that are easy to get wrong, stated here plainly: the API behind only 8 of the 13 commands was exercised, and only as direct HTTP requests — no koi- command was ever executed through XSIAM on this tenant. 5 commands change tenant state while the pack flags only 2 of them, which is read from the HTTP method each one issues, not from having run them. And of the 8 whose API was exercised, 7 are freely repeatable — koi-get-events is not.*

Pack documented: KOI 1.2.3 — demisto/content/Packs/Koi/Integrations/Koi/Koi.yml. Not the custom in-house pack (v1.3.0, 26 commands).